

Pj2 实验文档

一、 task1

1、模型结构

第一个 task 要求使用 HMM 隐式马尔科夫模型来解决 NER 命名实体识别任务。我自定义了类 HMM，初始化了五个参数，分别是标签集合 `state`，初始标签可能性 `initial_prob`，发射概率 `emission_prob`，词汇表 `vocab`，以及平滑参数 `smoothing`。

HMM 主要由两个函数 `train` 和 `viterbi_decode`，以及用于保存模型的函数 `save_model` 组成。其中，`train` 函数通过计数来计算 `initial_prob`、`transition_prob` 和 `emissionn_prob`。而 `viterbi_decode` 通过 `train` 函数计算好的参数来反向解码得到当前观测数据下每个字词对应的最佳标签。

2、模型参数与函数细节

在模型的初始化中，标签集合 `state`，初始标签可能性 `initial_prob` 和发射概率 `emission_prob` 是实现 HMM 必备的参数。词汇表 `vocab` 用来记忆所有学习过的字词。最后一个 `smoothing` 参数是不可或缺的超参数。以 `initial_prob` 为例，在训练时，有些标签从未出现在句子开头。若不加 `smoothing`，这些状态处在开头的概率就为 0，导致模型在推理时完全不考虑这些状态处在句子开头的情况，导致泛化能力变差。`Emission_prob` 中添加 `smoothing` 参数的原因是类似的。而在 `transition_prob` 中，若两个状态之间从未发生过转移，两者之间的转移概率就是 0。此时，在 `viterbi` 解码的过程中可能会出现概率链断裂，导致推理报错。在我的实验中，不添加 `smoothing` 参数时模型的确无法运行，这一现象印证了这个原因。这种平滑方式被成为“拉普拉斯平滑”。

由于 HMM 的概率是通过最大似然估计直接计算得出，不需要梯度下降，并且使用全量估计。在这方面，硬更新具有收敛速度快、实现简单、结果确定性强、计算效率高的优势，因此模型的参数更新我使用了硬更新的方式。相对地，软更新是每收到部分数就微调一次参数，其设计初衷在于适应数据流、在线学习或数据分布变化的场景，并不适合这个 task。

3、变量调整与结论

由于 HMM 完全通过计数的方式进行更新，因此变量只有一个——`smoothing` 参数。但是，处理 `label` 的方式也是决定正确率的因素之一。第一种方法是把所有可能的 `label` 包含在模型中，第二种是在遍历过程中动态加入 `label`，即只记录出现的 `label`。对于第一种方法，无论 `smoothing` 参数如何取舍，最终的 `f1 score` 均显著低于第二种（大约低了 0.2）。这是因为当 `label` 总数增多时，出现过的 `label` 概率会降低，未出现过的 `label` 概率会增大。而当 `label` 总数固定时，出现次数较多的 `label` 的概率会降低，出现次数少的概率会增大。这种概率稀释导致了最终结果差。最终测试中，中文在 `smoothing=1.1` 时取得最高的 `f1 score`，达 0.8912，英文在 `smoothing=0.01` 时取得最高的 `f1 score`，达 0.7876。这可以得出 `label` 数量越少时，`smoothing` 参数应该越小的结论。

根据我的理解，这种关系和学习率与任务难度的关系比较像。从 `pj1` 中可以得出任务越简单，学习率可以越大，任务越难，学习率需要越小的结论。类似地，在 `Label` 数量较少时，容易出现某种特定标签之间的转移次数显著高于其他转移的情况，模型就需要更细致地区分不同的转移方式，可以视作任务难度的增加。这时就需要更小的 `smoothing`，通过更细致的平滑方式来更好地完成任务。

4、对于 HMM 的理解

HMM 通过遍历整个数据集，计算出初始概率、转移概率和发射概率的出现次数，再分别除以对应的总次数，得到最终的三个矩阵。这种批量学习和频次统计的方式天然符合硬更新的特点，也具有简单便捷、易于复刻的特点。但是，这种单向的预测方式终究缺少了“上下文”中的“下”部分，不符合语言本身的特点。同时，HMM 的模板完全固定，不如 CRF 那般灵活，因此上限不高。最重要的是，这种基于全局统计、一次性归一化参数的方式会导致 `label bias`，即同一路径在出现次数相同时，样本量越小，其概率越高。这一概率往往与真实情况相差甚远，限制了 HMM 的准确率。`Label bias` 是 HMM 无法解决的设计缺陷，这也决定了 HMM 注定会被淘汰。

二、 task2

1、模型结构

我使用了自定义类 `CRF_Perceptron_Dynamic`，记录的主要参数是二元转移参数 `bigram_params` 和发射参数 `tag_word_params`，二者都是字典，用于存储不同类别的分数。同时，我将 `tag` 和 `word` 转为了具体的 `index`，方便模型计算。

2、模型参数与函数细节

`CRF_Perceptron_Dynamic` 类中的私有函数 `_get_bigram_score` 用于查询标签的转移分数，`_get_label_word_score` 用于查询标签和字词的发射分数。由于英文的 `f1-score` 分数较低，我进一步针对英文区分了首字母是否大写。`Viterbi` 解码部分与 HMM 类似，只是将概率连乘替换为了分数相加。尤其需要注意的是，普通的 HMM 中并不需要处理从开始状态到第一个状态和从最后一个状态到结束状态的转移，但是 CRF 需要。这是因为普通的 HMM 只是概率之间的乘积，并不关心句子什么时候开始和结束（初始概率仅仅是句子起始状态的可能性，模型并不知道“何时”应该开始）。在实验过程中，如果 CRF 不加入这两个状态转移的概率，最终的分数会偏低，因此，处理开始和结束是必要的。这体现了 CRF 更强的建模能力。

`Train_online` 函数是在线学习函数。在实验开始阶段，我在每个句子结束后都及时硬更新参数，然而实验分数较低，甚至越训练分数越低。这是由于①这种每次更新直接加减 1，缺乏学习率控制的方式，可能导致参数值过大，尤其在 5 轮训练后，高频特征（如 O 到 O 的转移）的权重可能过高，进而压制了其他特征。②同时，感知机只保留最后一次更新的参数，后期训练的更新可能覆盖早期学习的有效模式，导致过拟合训练数据。因此，我最终采用了平均参数的方式。实验表明，采用平均参数的方式有效解决了正确率低和越训练越差这两个问题。

3、参数调整与结论

num_iterations	final_score	num_iterations	final_score
1	0.818020575	1	0.925712
1	0.848272587	1	0.929053
1	0.856226998	1	0.927234
1	0.864874467	1	0.930494
1	0.865079365	1	0.930685
1	0.868157286	1	0.927719
1	0.869046212	1	0.929773
1	0.871206295	1	0.934306
1	0.876086828	1	0.934774
1	0.872302794	1	0.930695
1	0.875383888	1	0.935739
1	0.877777778	1	0.932611
1	0.878248002	1	0.93087
1	0.879210495	1	0.930033
1	0.872731572	1	0.932258
1	0.876469546		
1	0.875318066		
1	0.87416435		
1	0.878486998		
1	0.874055713		

左图为英文的正确率，右图为中文的正确率

在实验中，我先针对英语任务进行了 20 轮训练，在每一次训练后将参数和正确率记录在如图所示的 Excel 表格中。由图中数据可知，英语分词的 f1-score 最高达 0.88，相较于单纯使用 HMM 提升了约 0.1，非常可观。然而，在第八轮之后 f1-score 一直在 0.87 左右徘徊，表明模型已经饱和，训练次数过多。因此在中文任务上，我减小了训练轮数，一共训练 15 轮，最高达到 0.94 的分数，相较于 HMM 的 0.89 提升了 0.05，效果也很好。

4、对于 CRF 的理解

CRF 从 HMM 的有向图转变了无向图，支持对于不同任务人为定义多种类型的模板，使得 CRF 的评分系统能够考虑包括下文、单词特征、句子的开始与结束等更多因素。最重要的是，CRF 通过累加分数，而不是累乘当前的观测概率解决了 label bias 问题，极大地提升了模型的能力。在复杂度上，通过加入不同模板，极大地提升了 CRF 的上限；在性能上，CRF 相较于 transformer 架构更为轻量，在小样本下的结构化预测任务中仍然具有极高的效率。至今，CRF 仍然有着广泛的应用场景。

三、 task3

1、模型结构

在这个任务中，我自定义了类 NERDataset 用于加载数据，其中的 __getitem__ 函数将输入语句转换为对应的数字序号，并通过填充统一句子长度。类 PositionalEncoding 根据《Attention is all you need》的方式实现了位置编码。

CRF 类在 task2 中计算分数的基础上，增加了负对数似然的 loss 计算功能，其中私有函数 `_compute_partition_function` 计算全部可能路径的得分，`_compute_score` 计算真实标签的得分。这个 loss 函数符合 CRF 的逻辑，便于后续参数的自动更新。`Viterbi_decode` 实现 viterbi 解码功能。类 `TransformerCRF` 将字词转换为 embedding，并通过线性变换映射至 labels，输入到 CRF 类中。除此之外还有功能性函数 `build_vocab` 用于构建词汇表和标签集合，并建立映射关系；`evaluate_model` 用于计算 f1 score。

2、模型参数与函数细节

NERDataset 中，我在实验初期将句子统一截断到了 128 的长度，但是验证集中有长度超过 128 的，导致预测时出现匹配不上的情况。最后我采用了 256 的长度，因为 2 的整数次方便于 gpu 并行计算，同时能够涵盖所有句子的长度。

CRF 类中的 `_compute_partition_function` 函数接受的数据 `emissions` 表示一个 batch 中每句话在每个位置每个标签的得分。其中，`score` 矩阵将转移得分 `transitions` 矩阵并入上一个字词的 `emissions` 矩阵，并通过广播机制与当前字词相加得到了每一个 tag 下的得分，通过在上一个字词的维度上进行 `logsumup` 得到当前字词对应每一个标签的得分。因此，`score[i, j]` 表示第 i 个样本，从开始到当前时间步，以标签 j 结尾的所有可能路径的累积分数。当遍历完整个句子后再使用 `logsumup` 便得到一个 batch 内的总分数。

`_compute_score` 函数同样遍历整句话，不同的是通过 `gather` 函数只选择正确标签所对应的分数，最终返回形状为 `[batch_size]` 的向量，表示一个 batch 内每一句句子的总得分。

由于 viterbi 解码部分要求不能使用 pytorch，因此需要将张量转为 numpy 向量。同时，解码部分和 `_compute_partition_function` 一样需要遍历全部发射，不同之处在于前者只进行单纯累加并找出最大分数对应的标签，后者在每一步都会进行 `logsumup`。解码部分找到每一步最大分数的 index，加入到 `history` 矩阵中，在回溯时据此来选取最佳路径。

TransformerCRF 类中，我根据自身的设备能力使用了 `d_model=64`，`nhead=8`，`num_layers=4`，`max_len=256` 的形式。具体实现为将输入转换为 embedding，排除 padding 部分后进入 transformer 编码，最后通过线性层映射到不同的 tag。

3、参数调整与结论

在开始时，我按照《Attention is all you need》论文中的方式，将 `d_model` 设置为 512 维来设计 encoder 部分。但是这种高参数量使得我跑二十轮数据需要花 3 至 4 个小时，为调参带来了极大的不便，最终的分数也并不理想，中英文的分数都没有达到 70。这可能是由于参数量过大，导致训练次数过少时训练还不彻底。但是在这么小的数据集上训练时间已经过长，这使得我开始思考减少 `d_model` 数量。经过测试，我发现 `d_model` 设置为 64 时训练 20 轮的速度显著提升，英文大约使用 15 分钟，中文则是 25 分钟左右。同时，中文的 f1-score 来到 0.8605，英文来到 0.7782。

4、对于 transformer 的理解

从 HMM 到 CRF 再到 RNN、transformer，其实是模型寻找特征能力不断提升的过程。HMM 的建模非常简单，只包含初始概率，转移概率和发射概率。CRF 能够根据不同需求灵活指定不同的模板。但是本质上，这两种方法都是需要进行“特征工程”，即依赖人的先验经验来指定有较高概率发现有效特征的模板。历史上，人们找到的 CRF 模板高达一百多个，内卷现象十分严重。在当时的情景下，RNN 横空出世，通过不断更新 history 信息来有效应对长上下文的输入输出。同时，RNN 引入了不同的非线性变换，模型的表达能力远超前两者，有效地取代了繁琐的特征工程。然而 RNN 的线性变换随着序列长度的增加，注定导致距离越远的上文内容会被给予极小的权重，模型的长程依赖问题仍然存在。同时，gpu 技术迅猛发展，但是 RNN 却无法有效利用这种并行能力，人们迫切需要一种全新的方法。在这种背景之下，transformer 登上了历史舞台，也意味着人们彻底迈入深度学习阶段。通过位置编码，模型得以获取更长的上下文中字词的信息，从而有效解决了长距离定位问题，也将人们从窗口大小应该设置为多少的纠结之中解放了出来。进一步地，transformer 独特的 QKV 架构使得模型可以根据精确计算的相关性为每一个单词添加不同的权重，避免了死板地人为指定模板。同时，transformer 预训练得到的 embedding 可以直接提取出来，后续的实验不需重新学习。在利用 gpu 方面，相较于 RNN 依赖时间序列来处理信息，transformer 可以一次性针对整个文本进行处理，同步计算每一个字词的 QKV。Transformer 内部的参数更新也都依赖于矩阵乘法，这天然适配 gpu 的架构。而支持 batch 输入也能够有效利用 gpu。Transformer 的技术、特性都天然顺应历史潮流，取得了极大的成功。然而，transformer 对于算力的要求也较高，在数据量相对较少的情况下，CRF 仍让能够取得较好的成绩。在本次实验中，task3 使用了 transformer，但是中英文的分数都远远低于 task2 的分数，这说明在小数据集上，CRF 相较于 transformer 更为高效、准确。

四、 Bonus

1、函数调整

task2 单独要求使用 CRF 进行预测，其实并没有用到 gpu 的并行能力，因此我从一开始就没有使用 pytorch。因此，我着重讲述代码中我调整的部分。对于给出的模板，我定义了 parse_crf_template 来进行读取，使用 extract_features 来统一计算所有模板中的数据。这样一来，相较于 task2 中直接指定具体的参数更新，使用模板来进行更新可以减少代码，提升效率。同时，想要更改模板也只需要在 utf8 文件中进行删改即可，非常方便。

在 utf8 中，U 是 unigram 的缩写，B 则是 bigram 的缩写，在考虑了当前具体字词的前提下，前者只考虑当前字词的 label，后者考虑了当前字词和前一个字词的 label。由于加入了大量新的模板，训练时间大幅增加。在中文训练过程中，我使用了全部的 unigram 模板，1 轮分数便达到了 0.94，且第 3 轮仍然维持 0.94。由于 check 文件中计算 f1 分数是去掉了 O 标签后进行的，加上之后预测的正确率已经很高。而针对“实体”命名识别，0.94 的分数确实已经很高，因此我认为可以结束训练。在英文中，我去除了所有的跨 1 个字词的模板，因为跨越 1 个字词的关系性一般较少，去掉后对于分数影响也不大。但是由于英文分数比中文低，我同时使用了 unigram 和 bigram 的特征。最终，英文经过五轮训练得到了 0.83 的分数。在同时使用这

两种特征后，中文一轮达到了 0.94。

2、结论

相较于 task2 中的分数，使用更多 template 后，在相同训练时间下，中文分数持平，英文分数下降严重。在训练效率上，task2 中训练 20 轮的时间也在 25 分钟左右，但是 bonus 中训练一轮中文就要达到 50 分钟左右。这说明了更多的 template 并不意味着分数更好。我认为可能的原因如下：

首先，bonus 确实整合了更多的上下文在内，但本质上还是通过调整窗口大小来机械性地提取上下文特征，本质上是尝试用计算量换取准确度，不同于 transformer，方法上并没有本质上的区别。而一句话中一个字和前后两个字的关联往往并不存在普遍性，加入的新 template 不仅没有用，反而会导致模型记录下来的模板数量急速增多。然而每一个模板对应的分数和 0 相差无几，导致分数上的稀疏，如同不恰当的 smoothing 参数一样，增大了模型判别这些小分数之间的难度，从而导致准确率下降。

其次，针对英文，我在 task2 中专门做了大小写处理，这对于英文这种大小写敏感的语言非常重要，使得英文成绩一下子提升了 0.05 左右。因此，对于不同的文本，我们选择的模板应该重质而非量，好的模板才是提升分数的关键。

由此可见，盲目地堆砌算力并不是提升模型性能的最佳途径。这让我深刻反思：在当今社会，AI 巨头们不断争夺算力资源，模型参数规模也在不断膨胀。然而，正如开源且高效的 DeepSeek 等项目所展示的那样，算力的提升并不能替代方法上的创新。越是在算力日益充裕的时代，我们越应当关注算法与思路的优化，追求更高的效率与更优的效果，而不是陷入无止境的资源消耗与“堆料”竞赛。唯有如此，才能实现 AI 技术的可持续发展，真正推动行业进步。